Word Vectors, Semantics & Embeddings

COMP4153 Natural Language Processing

Kian Ming Lim

School of Computer Science
University of Nottingham Ningbo China

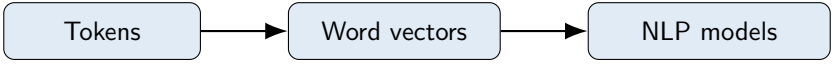
Where We Are in the Module

Last lecture

- NLP tasks and applications
- Text preprocessing pipeline
- Tokens, vocabulary and basic features

Today

- Represent word meaning as vectors
- Build embeddings from text statistics
- Understand Word2Vec, GloVe and contextual embeddings
- Build the mathematical foundation needed for later neural architectures



Learning Outcomes

By the end of this lecture, you should be able to:

- Explain why symbolic words need numerical representation.
- Compute similarity using vector operations and cosine similarity.
- Distinguish one-hot, count-based, prediction-based and contextual embeddings.
- Recognise limitations such as polysemy and bias.

Outline

1. Meaning as Geometry

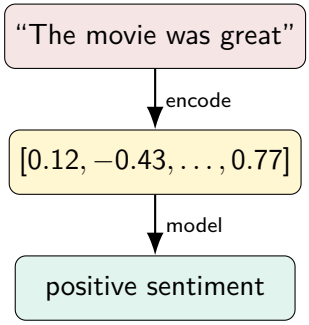
2. Vector Geometry

3. Learning Static Embeddings

4. From Static to Contextual Embeddings

Why Do We Need Word Vectors?

- Machine learning models expect numbers, not raw strings.
- A good representation should preserve useful semantic and syntactic information.
- Similar words should have similar representations.
- The representation should generalise beyond exact word matching.



Key question

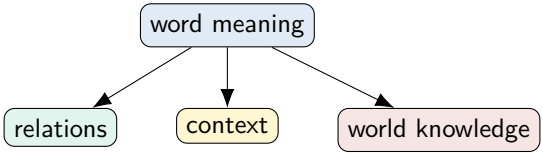
How can a vector capture part of a word's meaning?

Semantics: What Does "Meaning" Mean?

Lexical semantics

Lexical semantics studies the meaning of individual words and the relationships between words.

- **Synonymy:** words with similar meaning, e.g., *big* and *large*
- **Antonymy:** words with opposite meaning, e.g., *hot* and *cold*
- **Hyponymy:** "is-a" relationship, e.g., *sparrow* is a kind of *bird*
- **Polysemy:** one word with multiple meanings, e.g., *bank* as river side or financial institution



The Distributional Hypothesis

"You shall know a word by the company it keeps."

J. R. Firth

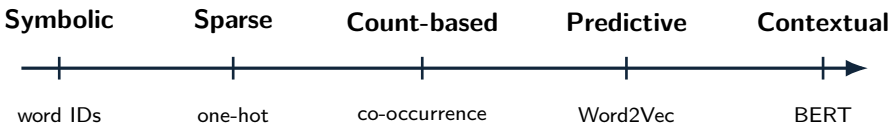
Core idea

Words that occur in similar contexts tend to have similar meanings.

Examples

- *doctor* and *nurse*: patient, hospital, medicine, clinic
- *king* and *queen*: palace, throne, crown, royal
- *delicious* and *tasty*: food, meal, restaurant, recipe

Representation Options: A Spectrum



Today's focus

How can we move from words as tokens to vectors that capture meaning?

Symbolic Representation: Word IDs

Before using vectors, NLP systems often first convert words into integer IDs.

Word	ID
cat	0
dog	1
bank	2
river	3
loan	4

A vocabulary maps each word type to a unique integer.

Why useful?

Integer IDs allow words to be stored, indexed and processed efficiently by a computer.

Main limitation

The numbers themselves do not represent meaning:

$$\mathrm{ID}(\textit{dog}) = 1, \quad \mathrm{ID}(\textit{bank}) = 2$$

This does not mean that *dog* and *bank* are semantically close.

One-Hot Encoding

Suppose the vocabulary is $V = \{\text{cat}, \text{dog}, \text{bank}, \text{river}, \text{loan}\}$.

Word	cat	dog	bank	river	loan
cat	1	0	0	0	0
dog	0	1	0	0	0
bank	0	0	1	0	0
river	0	0	0	1	0
loan	0	0	0	0	1

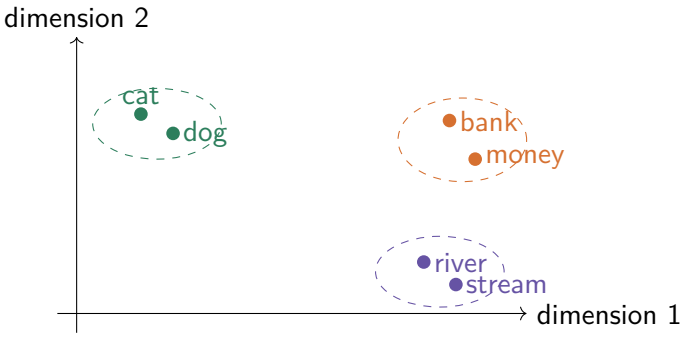
Advantages

Simple, deterministic, easy to implement.

Problem

All different words are equally unrelated: $\mathbf{x}_{cat}^{\top}\mathbf{x}_{dog} = 0$.

Dense Embeddings: The Target Representation



Goal

Similar or related words should be nearby in the vector space.

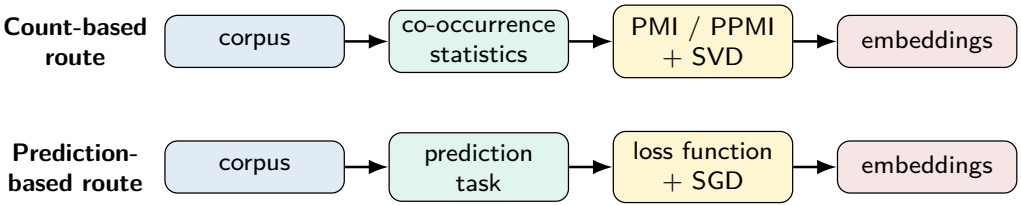
Two Families of Static Word Embeddings

Count-based embeddings

- Build a word-context co-occurrence matrix.
- Reweight counts using PMI or PPMI.
- Reduce dimensionality using SVD.

Prediction-based embeddings

- Learn vectors by predicting context words.
- Optimise a loss function.
- Use SGD to update vectors.



Outline

1. Meaning as Geometry

2. Vector Geometry

3. Learning Static Embeddings

4. From Static to Contextual Embeddings

Vector Basics

A word embedding maps each word w to a vector:

$$\mathbf{v}_w = [v_1, v_2, \ldots, v_d] \in \mathbb{R}^d$$

Example

$$\mathbf{v}_{cat} = [0.2, 0.7, -0.1, 0.4]$$

$$\mathbf{v}_{dog} = [0.3, 0.6, -0.2, 0.5]$$

Interpretation

- Dimensions are learned features.
- A single dimension may not be easy to interpret, but the overall pattern of values is meaningful.

Dot Product

For two vectors $\mathbf{x}$ and $\mathbf{y}$:

$$\mathbf{x}^T\mathbf{y} = \sum_{i=1}^{d} x_i y_i$$

Example:

$$\mathbf{x} = [1, 2, 0], \quad \mathbf{y} = [2, 1, 1]$$
$$\mathbf{x}^T\mathbf{y} = 1 \times 2 + 2 \times 1 + 0 \times 1 = 4$$

What does it measure?

The dot product gives a score showing how much two vectors align with each other. If two vectors have large values in similar dimensions, the dot product becomes large.

In word embeddings, this can indicate that two words are related or similar.

Note:

The dot product is affected by both **direction** and **length**. A long vector can produce a large dot product even if the direction is not very similar.

Cosine Similarity

Cosine similarity measures the angle between two vectors:

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \, \|\mathbf{y}\|}$$

Range

- 1: same direction
- 0: orthogonal
- -1: opposite direction

Why use cosine?

- Focuses on direction rather than length.
- Useful for high-dimensional sparse vectors.
- Allows us to find nearest neighbours in the embedding space.

Example: Cosine Similarity

Consider three 2D vectors:

$$\mathbf{v}_{cat} = [2, 1], \quad \mathbf{v}_{dog} = [2, 2], \quad \mathbf{v}_{car} = [0, 3]$$

cat vs dog

$$\mathbf{v}_{cat}^T \mathbf{v}_{dog} = 2 \cdot 2 + 1 \cdot 2 = 6$$

$$\|\mathbf{v}_{cat}\| = \sqrt{5}, \quad \|\mathbf{v}_{dog}\| = \sqrt{8}$$

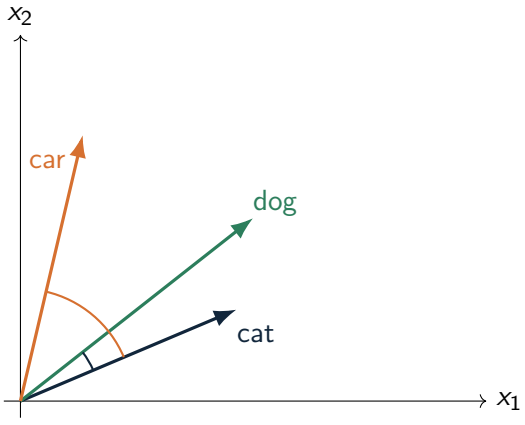
$$\cos = \frac{6}{\sqrt{5}\sqrt{8}} \approx 0.95$$

cat vs car

$$\mathbf{v}_{cat}^T \mathbf{v}_{car} = 3$$

$$\cos = \frac{3}{\sqrt{5} \cdot 3} \approx 0.45$$

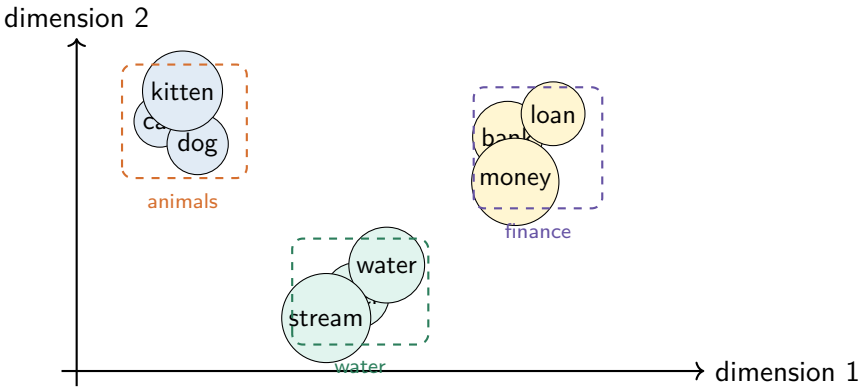
Visualisation: Cosine as Angle



Interpretation

The angle between cat and dog is smaller than the angle between cat and car, so cat is closer to dog.

Vector Space View of Semantics



Geometric intuition

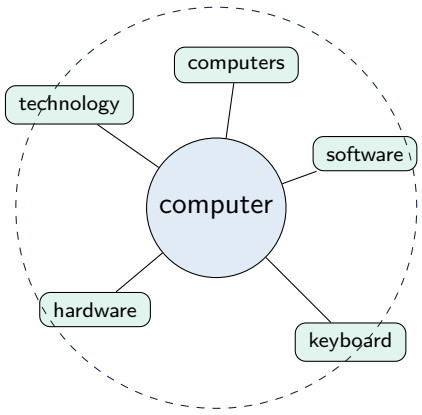
Semantic similarity becomes spatial closeness in a learned vector space.

Nearest Neighbours in Embedding Space

Example query

Given a vector for *computer*, find words with the highest cosine similarity.

Nearest word	Cosine
software	0.78
computers	0.76
technology	0.72
hardware	0.70
keyboard	0.63



Caution

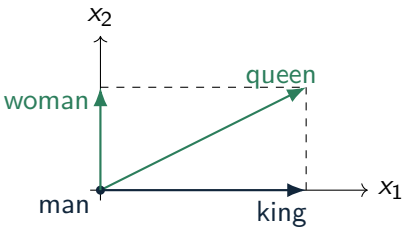
Nearest neighbours reveal what the model learned from the corpus, not always what is correct or desirable.

Vector Arithmetic and Analogies

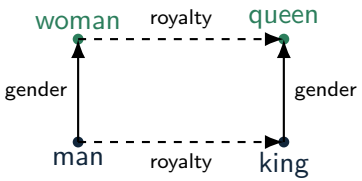
A famous property of word embeddings is approximate linear structure:

$$\mathbf{v}_{king} - \mathbf{v}_{man} + \mathbf{v}_{woman} = \mathbf{v}_{king} + (\mathbf{v}_{woman} - \mathbf{v}_{man}) \approx \mathbf{v}_{queen}$$

Word vectors from origin



Relationship directions



Caution

Analogies are interesting, but they are not always robust and can reveal bias.

Outline

1. Meaning as Geometry

2. Vector Geometry

3. Learning Static Embeddings

4. From Static to Contextual Embeddings

Tiny Corpus

We start with a very small example corpus.

1. cats chase mice
2. dogs chase cats
3. dogs eat food

Vocabulary

$$V = \{\text{cats}, \text{dogs}, \text{chase}, \text{mice}, \text{eat}, \text{food}\}$$

Window size $= 1$

For each centre word, count the word immediately before and after it.

Example

In "cats chase mice", the context of *chase* is $\{cats, mice\}$.

Building a Complete Co-occurrence Matrix

Rows are target words; columns are context words. With window size 1, we count neighbouring words on the left and right.

Target \ Context	cats	dogs	chase	mice	eat	food
cats	0	0	2	0	0	0
dogs	0	0	1	0	1	0
chase	2	1	0	1	0	0
mice	0	0	1	0	0	0
eat	0	1	0	0	0	1
food	0	0	0	0	1	0

Interpretation

Each row is a sparse count vector. It represents a target word by the neighbouring words it appears with.

Sparse Count Vectors

Using the column order:

$$[\text{cats}, \text{dogs}, \text{chase}, \text{mice}, \text{eat}, \text{food}]$$

Example vectors

$$\mathbf{x}_{\text{cats}} = [0, 0, 2, 0, 0, 0]$$

$$\mathbf{x}_{\text{dogs}} = [0, 0, 1, 0, 1, 0]$$

$$\mathbf{x}_{\text{chase}} = [2, 1, 0, 1, 0, 0]$$

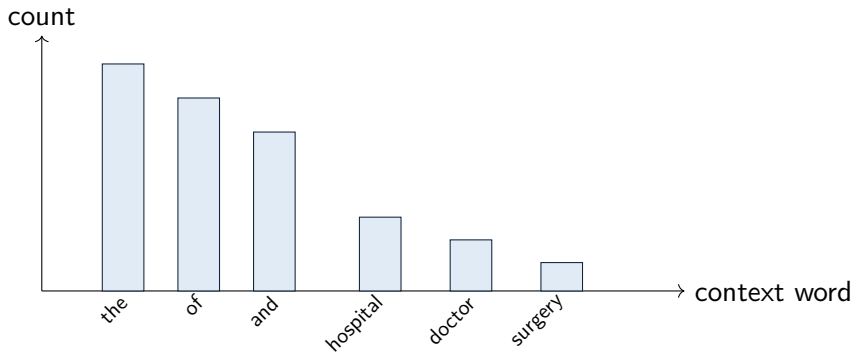
$$\mathbf{x}_{\text{food}} = [0, 0, 0, 0, 1, 0]$$

Problems

- High-dimensional for large vocabularies
- Mostly zeros
- Raw counts favour frequent words
- Hard to generalise

Raw Counts Can Be Misleading

Frequent words such as *the*, *of*, *and* appear near many words.



Question

Does a large raw count always mean a context word is semantically informative?

Pointwise Mutual Information (PMI)

PMI measures how much more often a word and context occur together than expected by chance:

$$PMI(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$$

Interpretation

- Positive: w and c co-occur more than expected.
- Zero: independent.
- Negative: co-occur less than expected.

PPMI

Positive PMI keeps only positive values:

$$PPMI(w, c) = \max(PMI(w, c), 0)$$

Example: PMI

Suppose we count 100 word-context pairs.

Observed counts

- count(*bank*, *money*) = 5
- count(*bank*) = 10
- count(*money*) = 8
- total pairs = 100

PMI calculation

$$P(w, c) = \frac{5}{100} = 0.05$$

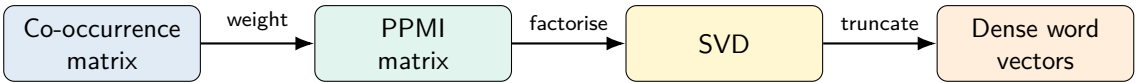
$$P(w)P(c) = \frac{10}{100}\frac{8}{100} = 0.008$$

$$PMI = \log_2 \frac{0.05}{0.008} \approx 2.64$$

Interpretation

bank and *money* co-occur much more often than chance.

From Count Matrix to Dense Embeddings



Purpose of Singular Value Decomposition (SVD)

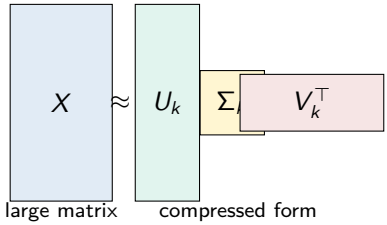
Find a lower-dimensional representation that preserves major patterns of co-occurrence.

Dimensionality Reduction

A count matrix can be compressed into dense word vectors:

$$X \approx U_k \Sigma_k V_k^\top$$

- X: original co-occurrence or PPMI matrix
- U_k: word vectors in a lower-dimensional space
- Σ_k: importance of each dimension
- $V_k^\top$: context vectors
- k: number of dimensions



Main idea

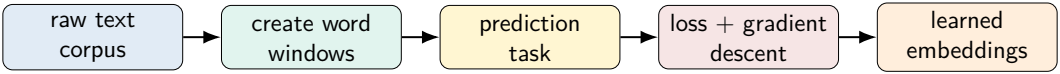
Keep only the most important k dimensions, instead of using the full large high dimensional matrix.

Prediction-Based Embeddings

Learn word vectors by solving a self-supervised prediction task.

Core idea

Use raw text to create prediction examples. The model learns embeddings that help predict missing or neighbouring words.



- No manual labels are needed.
- Training examples come from word windows.
- A loss function measures prediction error.
- Embeddings are updated during training.
- After training, keep the learned embedding matrix.
- Word2Vec is a classic example.

Word2Vec: Two Training Objectives

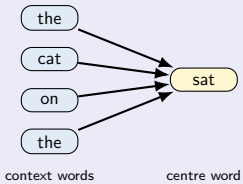
window size = 2

the	cat	sat	on	the	mat

centre word

Continuous Bag of Words (CBOW)

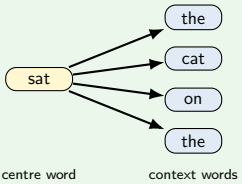
Predict the centre word from surrounding context words.



$\{the, cat, on, the\} \rightarrow sat$

Skip-gram

Predict surrounding context words from the centre word.



$sat \rightarrow \{the, cat, on, the\}$

Word2Vec: Overview

Word2Vec is a framework for learning dense word vectors from large unlabelled text. (Mikolov et al. 2013)

$$\text{word} \longrightarrow \text{vector representation}$$

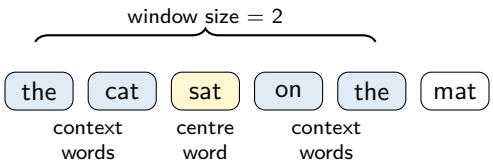
Main idea

Use the context around a word to learn its meaning.

- Start with a large text corpus: a long sequence of words.
- Every word in a fixed vocabulary is represented by a vector.
- Move through each position t in the text.
- At each position, identify a **centre word** c and its **context words** o.
- Use vector similarity for c and o to calculate the probability of o given c (or vice versa).
- Keep adjusting the word vectors to maximize the probability.

Word2Vec: Overview

Word2Vec learns word vectors by using neighbouring words as supervision.



Learning process

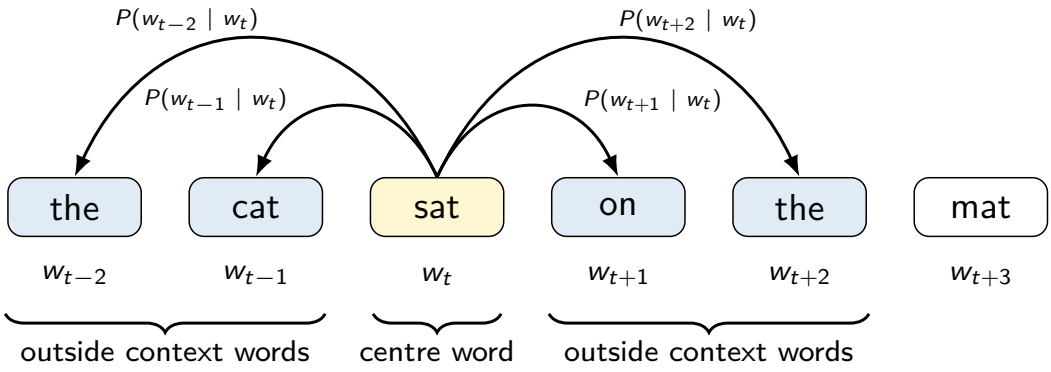
- Initialise word vectors randomly.
- Predict words in the local window.
- Measure prediction error using a loss function.
- Update vectors using gradient descent.

Why it works

Words that occur in similar contexts receive similar updates, so their vectors become closer in the embedding space.

Word2Vec: Context Window

Example window of size 2 for computing $P(w_{t+j} \mid w_t)$

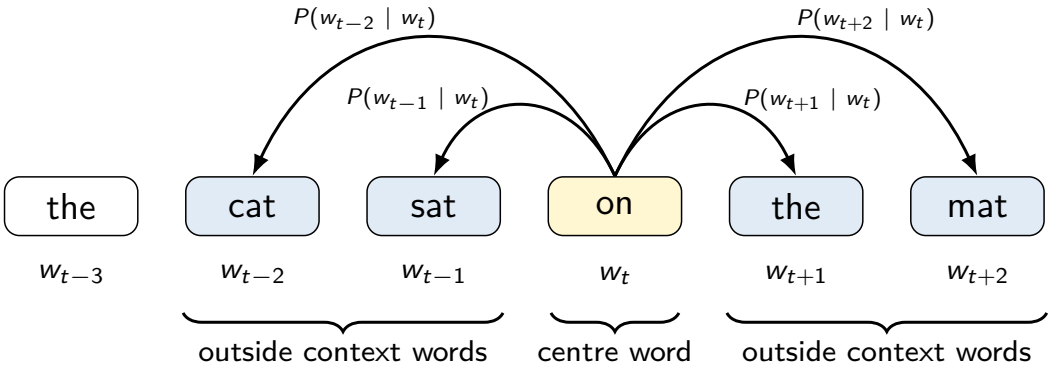


Window size = 2

For the centre word *sat*, the context words are $\{the, cat, on, the\}$.

Word2Vec: Sliding the Context Window

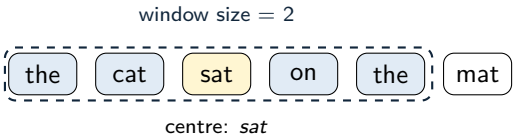
The window moves one step: centre word changes from *sat* to *on*



Window size = 2

For the centre word *on*, the context words are $\{cat, sat, the, mat\}$.

A sentence produces many training examples



Skip-gram style pairs

Using centre word → context word:

(sat, the)

(sat, cat)

(sat, on)

(sat, the)

Learning intuition

The model increases the probability of true context words:

$$P(cat \mid sat), \quad P(on \mid sat)$$

and decreases the probability of unlikely words:

$$P(banana \mid sat), \quad P(finance \mid sat)$$

Word2Vec: Objective Function

For each position $t = 1, \ldots, T$, predict context words within a fixed window size m. Data likelihood:

$$L(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} \mid w_t; \theta)$$

where:

- w_t: centre word at position t
- w_{t+j}: context word around w_t
- θ: all parameters to be learned

The objective function $J(\theta)$ is the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} \mid w_t; \theta)$$

Minimising objective function $\Leftrightarrow$ Maximising likelihood of observed context words

Word2Vec: Objective Function

We use Skip-gram to explain the Word2Vec training objective: predict context words from a centre word. We want to minimize the objective function:

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-m\le j\le m\\ j\neq 0}} \log P(w_{t+j} \mid w_t;\theta)$$

Question: How do we calculate $P(w_{t+j} \mid w_t;\theta)$?

For each word w, we learn two vectors:

$\mathbf{v}_w$: centre-word vector

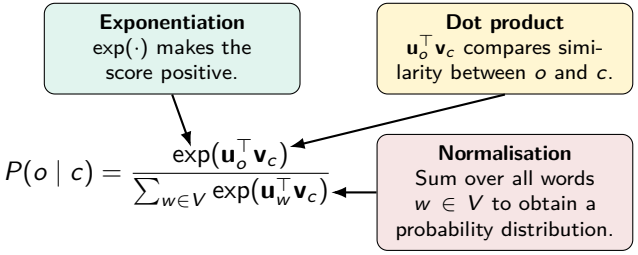
$\mathbf{u}_w$: context-word vector

For a centre word c and context word o:

$$P(o \mid c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w\in V}\exp(\mathbf{u}_w^\top \mathbf{v}_c)}$$

Word2Vec: Prediction Function

For a centre word c and an outside context word o:



- Larger $\mathbf{u}_o^\top \mathbf{v}_c$ means context word o is more compatible with centre word c, i.e. larger probability.
- Softmax maps arbitrary scores into probabilities.
- The denominator is expensive because it sums over the whole vocabulary.
- For large vocabularies, computing the full softmax is expensive. Word2Vec often uses negative sampling.

Why Naive Softmax Is Expensive

$$P(o \mid c) = \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^T \mathbf{v}_c)}$$

If vocabulary size is 100,000

For every centre word, softmax requires scoring all 100,000 possible outside words, very heavy computation.

Too slow

Most words are not relevant to the current training window, yet they appear in the denominator.

We need a cheaper training objective!

Negative Sampling

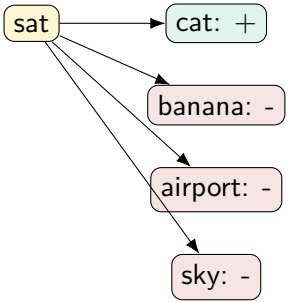
Instead of predicting the full vocabulary, train the model to distinguish real pairs from randomly sampled noise pairs.

Positive pair

(sat, cat) occurs in the corpus.

Negative pairs

$(sat, banana)$, $(sat, airport)$, (sat, sky) are sampled as noise.



Intuition: pull real word-context pairs together, push random pairs apart.

Negative Sampling: Key Idea

Instead of predicting the entire vocabulary, train binary classifiers for word pairs.

Positive pair

A real centre-context pair from the corpus.

$$(sat, cat) \rightarrow 1$$

Negative pairs

Randomly sampled noise words.

$$(sat, banana) \rightarrow 0$$

$$(sat, finance) \rightarrow 0$$

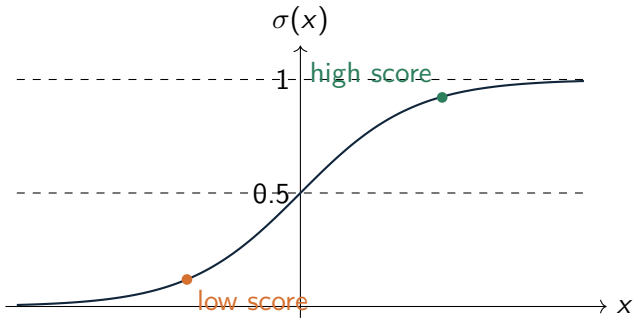
Learning objective

Make real pairs score high and random pairs score low.

Sigmoid Function

Negative sampling uses the logistic sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Negative Sampling Loss

For one positive pair (c, o) and K negative samples $k_1, \ldots, k_K$:

$$J = -\log \sigma(\mathbf{u}_o^T \mathbf{v}_c) - \sum_{i=1}^{K} \log \sigma(-\mathbf{u}_{k_i}^T \mathbf{v}_c)$$

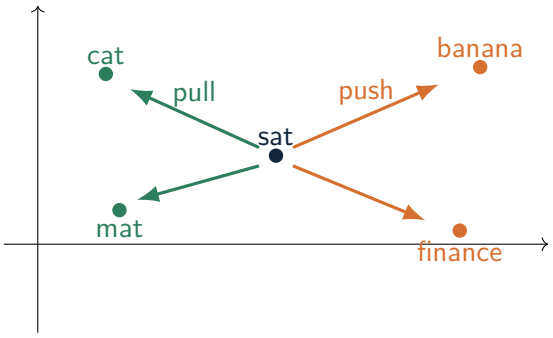
Interpretation

- First term rewards high score for the true outside word.
- Second term rewards low scores for random negative words.

Efficiency

Only $K + 1$ outside vectors are updated, not the whole vocabulary.

Push and Pull View of Training



After many updates

Words that appear in similar contexts are pulled into similar regions of the vector space.

Loss Function: Negative Log-Likelihood

For one positive pair (c, o), the loss is:

$$J = -\log P(o \mid c)$$

Good prediction

If $P(o \mid c) = 0.90$:

$$J = -\log(0.90) = 0.105$$

Low loss.

Bad prediction

If $P(o \mid c) = 0.05$:

$$J = -\log(0.05) = 2.996$$

High loss.

Key idea

Training tries to make the observed context words more probable.

Numerical Example: Softmax and Loss

For centre word c and outside word o, let score s be the dot product:

$$s(o, c) = \mathbf{u}_o^T \mathbf{v}_c$$

Suppose the centre word is *sat*. The model scores three candidate outside words.

Outside word	cat	mat	banana
Score s	2.0	1.0	0.0
$\exp(s)$	7.39	2.72	1.00
$P(w \mid sat)$	0.67	0.24	0.09

If the true outside word is *cat*

$$J = -\log(0.67) \approx 0.40$$

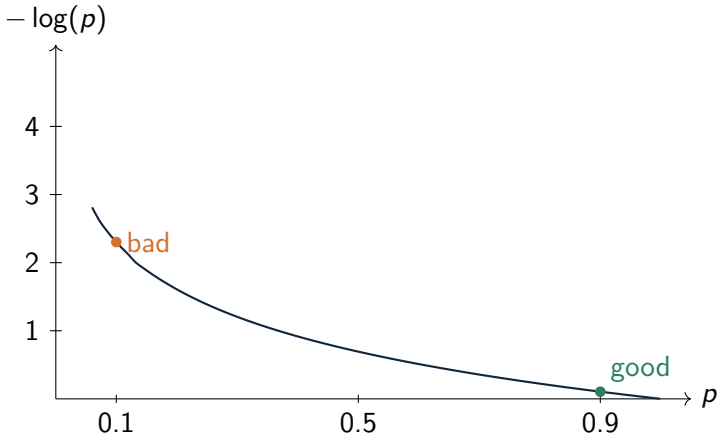
Low loss.

If the true outside word is *banana*

$$J = -\log(0.09) \approx 2.41$$

High loss.

Visualising the Negative Log Loss



Takeaway

The loss becomes very large when the model assigns low probability to the correct word.

Training Word2Vec: Optimising Parameters

To train the model, we gradually adjust all parameters θ to minimise the loss.

- θ represents **all learnable parameters**.
- In Word2Vec, every word has two vectors: $\mathbf{v}_w$ (centre-word vector), $\mathbf{u}_w$ (context-word vector)
- For vocabulary size $|V|$ and embedding dimension d:

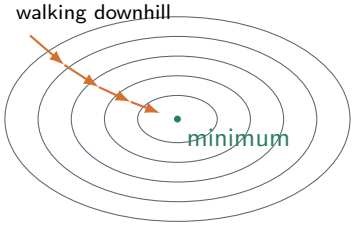
$$\theta \in \mathbb{R}^{2d|V|}$$

- We update these vectors using gradients:

$$\theta \leftarrow \theta - \alpha \nabla_\theta J(\theta)$$

- Training computes gradients for the word vectors and updates them to reduce the objective function.

$$\theta = \begin{bmatrix} \mathbf{v}_a \\ \vdots \\ \mathbf{v}_{\text{zebra}} \\ \mathbf{u}_a \\ \vdots \\ \mathbf{u}_{\text{zebra}} \end{bmatrix} \in \mathbb{R}^{2d|V|}$$



What Does the Gradient Do?

The gradient tells us how to change each parameter to reduce the loss.

$$\theta^{new} = \theta^{old} - \alpha \nabla_\theta J(\theta)$$

For real context pairs

Move vectors to increase their dot product.

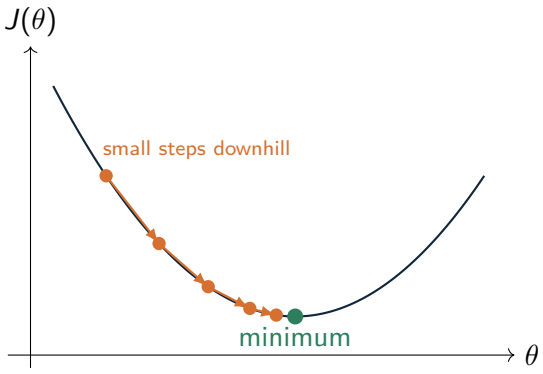


For negative pairs

Move vectors to decrease their dot product.



Gradient Descent Intuition



Algorithm

1. Compute current loss.
2. Compute gradient.
3. Take a small step downhill.
4. Repeat many times.

Important

Neural NLP losses are usually not simple convex bowls, but SGD often works well in practice.

Stochastic Gradient Descent (SGD)

Full objective over the whole corpus:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J_t(\theta)$$

Problem

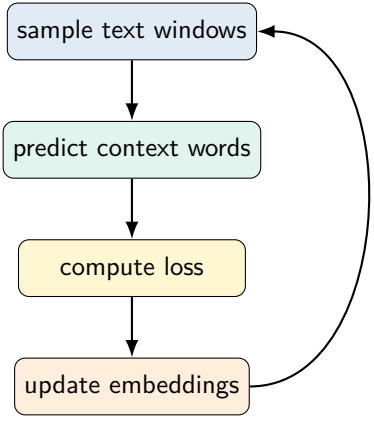
For a large corpus, computing the full gradient after every update is too expensive.

SGD solution

Repeatedly sample one window or a mini-batch of windows and update immediately:

$$\theta \leftarrow \theta - \alpha \nabla_\theta J_{batch}(\theta)$$

Mini-Batch Training Loop

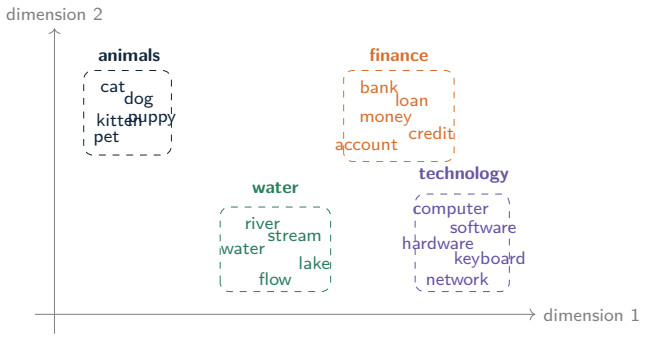


Practical interpretation

Every mini-batch slightly reshapes the word vector space.

Visualising Word2Vec Embeddings

Word2Vec learns a high-dimensional vector for each word. We can project these vectors into 2D using PCA, t-SNE or UMAP.



Key idea

Words with similar contexts are mapped to nearby points in the embedding space.

Try it yourself

Explore real word embeddings using TensorFlow Embedding Projector:

`https://projector.tensorflow.org/`

Static Embeddings: Strengths and Weaknesses

Strengths

- Efficient to train and use
- Good for similarity, clustering and classical NLP pipelines
- Can be trained on domain-specific corpora
- Interpretable through nearest neighbours

Weaknesses

- One vector per word type
- Poor handling of polysemy
- Difficulty with out-of-vocabulary words
- Sensitive to corpus bias and preprocessing
- Limited compositional understanding

Outline

1. Meaning as Geometry

2. Vector Geometry

3. Learning Static Embeddings

4. From Static to Contextual Embeddings

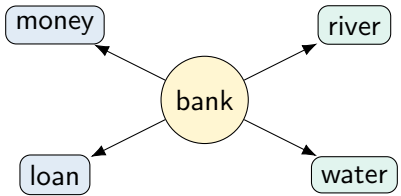
The Polysemy Problem

Sentence 1

She deposited money in the **bank**.

Sentence 2

They sat by the river **bank**.



Issue

A static embedding gives *bank* one vector, mixing multiple senses into one representation.

Same word, different sentence, different vector.

Financial context

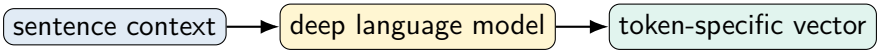
The bank approved the loan.

$$\mathbf{h}_{bank}^{(financial)}$$

River context

The boat reached the bank.

$$\mathbf{h}_{bank}^{(river)}$$



Static vs Contextual Embeddings

	Static embedding	**Contextual embedding**
Representation	One vector per word type	One vector per token occurrence
Examples	Word2Vec, GloVe, fastText	ELMo, BERT, RoBERTa, GPT-style models
Handles polysemy?	Limited	Much better
Computational cost	Low	Higher
Best use	Classical ML, efficient features	Modern transfer learning, fine-tuning

Practical reality

Most modern NLP systems use contextual representations, but static embeddings remain valuable for understanding, efficiency and specialised applications.

References

- Jurafsky and Martin, *Speech and Language Processing*, chapters on vector semantics and embeddings.
- Mikolov et al. (2013), Efficient Estimation of Word Representations in Vector Space.